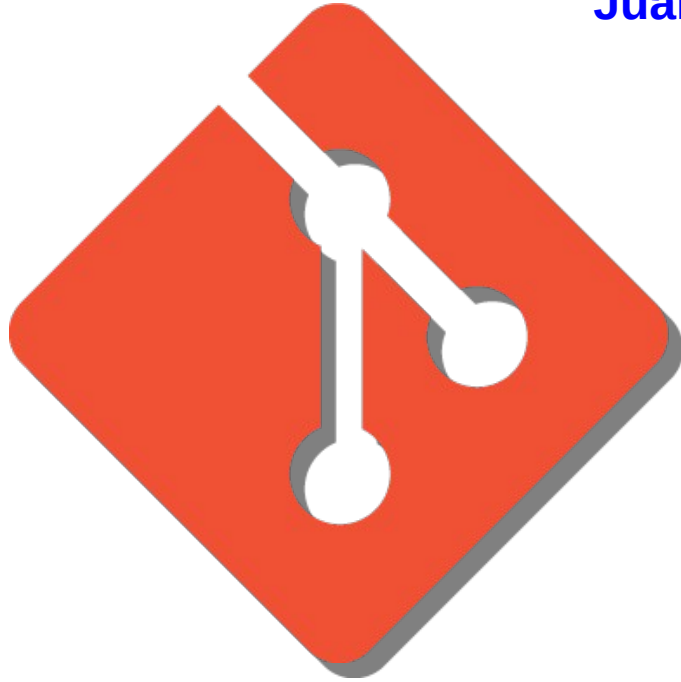


Control de versions



Juanma Sánchez

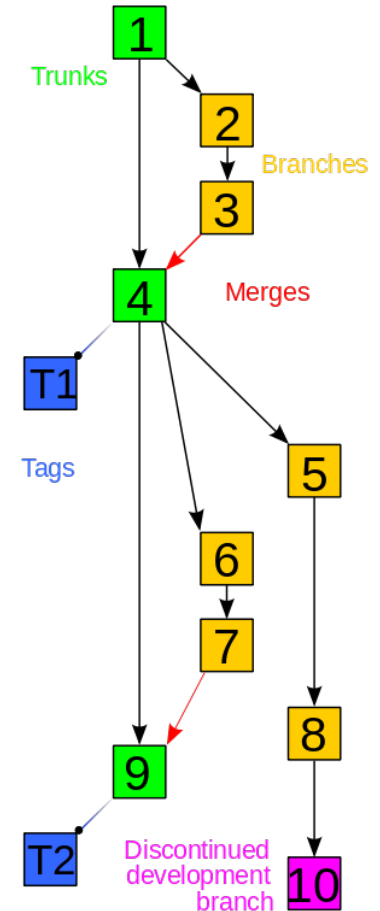
git

Introducció

- El control de versions és la gestió dels canvis que es produeixen en el programari o dades i que és gestionat per un VCS (Version Control System)
- Cada canvi té associat:
 - ♦ Nom del canvi
 - ♦ Autor del canvi
 - ♦ Data i hora
 - ♦ Diferències del canvi respecte al que està guardat
 - ♦ Motiu per el qual es fa el canvi

Definicions

- El desenvolupament en equip pot dur a que la versió principal del codi (**main**) avanci en paral·lel amb grups separats (**branch**)
- Quan diferents persones fan canvis al mateix arxiu s'han de conciliar (**merge**) o també al ajuntar branques
- Quan és una fita (**tag**)



Models clàssics

- Històricament el controls de versions han estat centralitzats i tenen:
 - **Operacions atòmiques** → cas contrari la barreja de canvis parcials duu a corrupció de fitxers
 - **Bloqueig de fitxer** → per evitar merges
 - «**Merge**» de versions → només viable en text
 - «**Baselines**», «**merges**» i «**tags**» → Com hem vist ja

Model descentralitzat

- Utilitzen *peer to peer* en lloc del model client servidor
- Cada desenvolupador té una còpia completa del projecte i treballa sobre ella en un model de «bona fide» (bona fe)
- La sincronització es fa intercanviant «patches» entre repositoris

Conseqüències de descentralitzar

- No hi ha una còpia central, tot són còpies de treball
- La majoria d'operacions són molt ràpides ja que es fan en local
- Només cal xarxa per fer «**push**» o «**pull**»
- Cada còpia de treball és un backup remot del codi base

Diferents controls de versions

Distribuïts



Client-servidor



Repositoris públics Git



- Escrit en Ruby on Rails
- Comprat per Msoft en 2018
- Dona:
 - Repositori al núvol
 - Wiki
 - Pàgina Web
 - Gràfic de historial
 - Gestor Kanban
 - Gestió tipus DevOps



- Escrit en Ruby
- Originalment open source
- Dona:
 - Repositori al núvol
 - Wiki
 - Pàgina Web
 - Gestor Kanban
 - Gestió tipus DevOps